

Tutorial on Block Ciphers

Shibam Ghosh

June 10, 2026

Department of CSE, IIT Hyderabad

Table of contents

1. Introduction
2. The Advanced Encryption Standard (AES)
3. The Feistel Structure and DES

Introduction

Symmetric-Key Cryptography

Symmetric-Key Cryptosystem

$\mathcal{P}$

$\mathcal{K}$

$\mathcal{C}$

$\mathcal{E}$

$\mathcal{D}$

$$\forall P \in \mathcal{P}, K \in \mathcal{K}, \\ \mathcal{D}_K(\mathcal{E}_K(P)) = P$$

e.g., Block ciphers, stream ciphers, hash functions, MACs, ...

Symmetric-Key Cryptography

Symmetric-Key Cryptosystem

$\mathcal{P}$

$\mathcal{K}$

$\mathcal{C}$

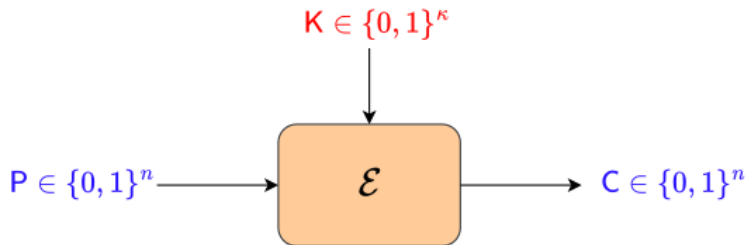
$\mathcal{E}$

$\mathcal{D}$

$$\forall P \in \mathcal{P}, K \in \mathcal{K}, \\ \mathcal{D}_K(\mathcal{E}_K(P)) = P$$

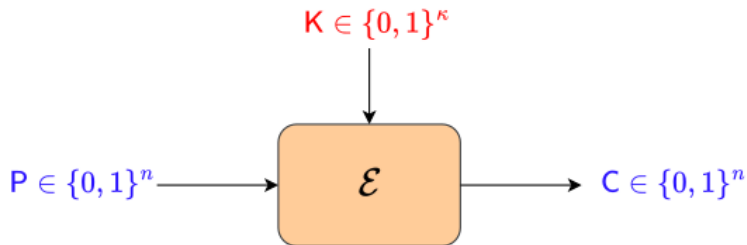
e.g., Block ciphers, stream ciphers, hash functions, MACs, ...

Block Cipher



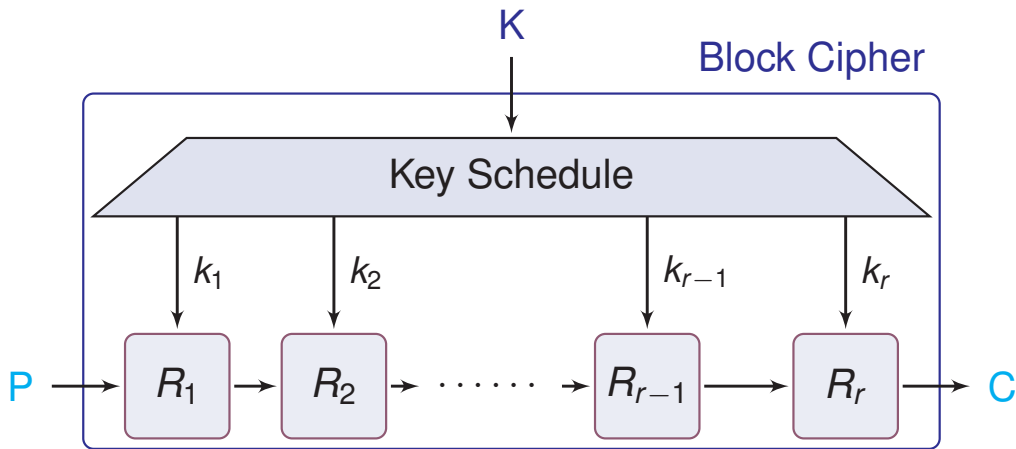
- ▶ $E : \{0, 1\}^n \times \{0, 1\}^\kappa \rightarrow \{0, 1\}^n$ s.t. $C = \mathcal{E}(P, K)$
- ▶ $E_K : \{0, 1\}^n \rightarrow \{0, 1\}^n$ s.t. $C = \mathcal{E}_K(P)$

Block Cipher

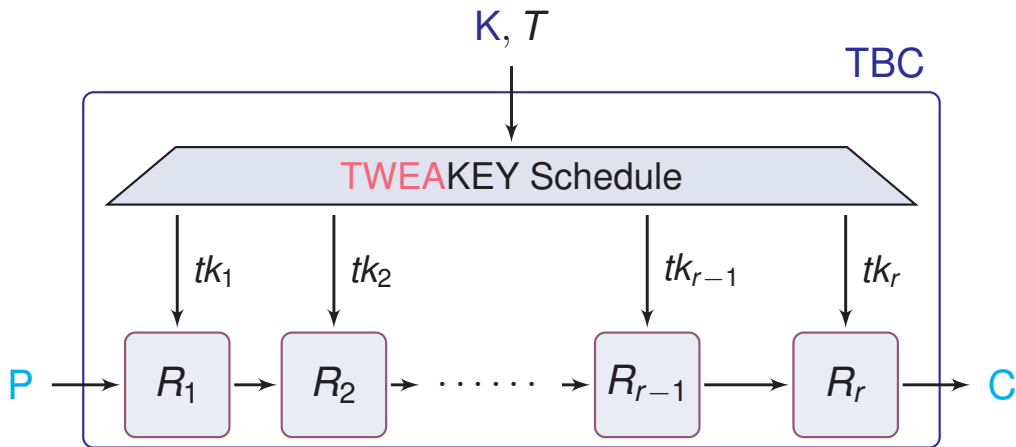


- ▶ $E : \{0, 1\}^n \times \{0, 1\}^\kappa \rightarrow \{0, 1\}^n$ s.t. $C = E(P, K)$
- ▶ $E_K : \{0, 1\}^n \rightarrow \{0, 1\}^n$ s.t. $C = E_K(P)$

Iterated Block Cipher



Iterated Tweakable Block Cipher with TWEAKEY Schedule



The Advanced Encryption Standard (AES)

The Advanced Encryption Standard

- ▶ **The Competition of (1997)** by NIST (National Institute of Standards and Technology) for a highly secure, efficient, and royalty-free cipher
- ▶ **The Finalists (1999):** Out of 15 initial submissions, 5 finalists were selected: MARS, RC6, **Rijndael**, Serpent, and Twofish.
- ▶ **The Winner (2000): Rijndael**, designed by Belgian cryptographers Joan Daemen and Vincent Rijmen
 - ▷ Excellent balance of security and performance
 - ▷ Flexibility across hardware and software
- ▶ **Standardization (2001):** Officially published as FIPS PUB 197

The Advanced Encryption Standard

- ▶ **The Competition of (1997)** by NIST (National Institute of Standards and Technology) for a highly secure, efficient, and royalty-free cipher
- ▶ **The Finalists (1999):** Out of 15 initial submissions, 5 finalists were selected: MARS, RC6, **Rijndael**, Serpent, and Twofish.
- ▶ **The Winner (2000): Rijndael**, designed by Belgian cryptographers Joan Daemen and Vincent Rijmen
 - ▷ Excellent balance of security and performance
 - ▷ Flexibility across hardware and software
- ▶ **Standardization (2001):** Officially published as FIPS PUB 197

The Advanced Encryption Standard

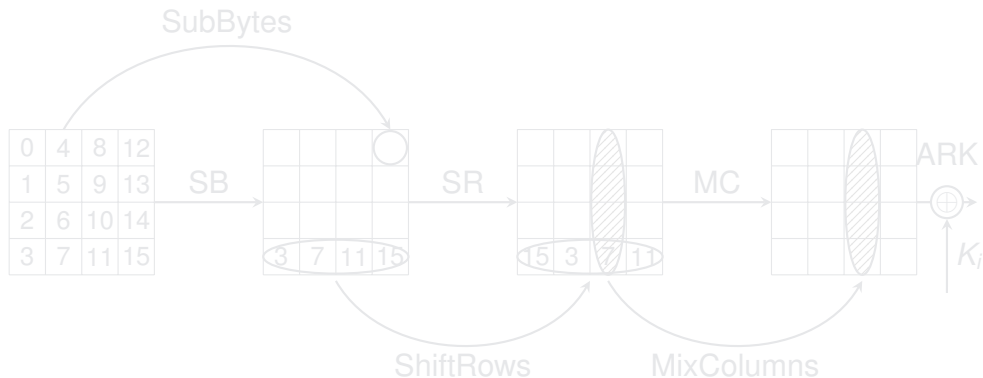
- ▶ **The Competition of (1997)** by NIST (National Institute of Standards and Technology) for a highly secure, efficient, and royalty-free cipher
- ▶ **The Finalists (1999):** Out of 15 initial submissions, 5 finalists were selected: MARS, RC6, **Rijndael**, Serpent, and Twofish.
- ▶ **The Winner (2000): Rijndael**, designed by Belgian cryptographers Joan Daemen and Vincent Rijmen
 - ▷ Excellent balance of security and performance
 - ▷ Flexibility across hardware and software
- ▶ **Standardization (2001):** Officially published as FIPS PUB 197

The Advanced Encryption Standard

- ▶ **The Competition of (1997)** by NIST (National Institute of Standards and Technology) for a highly secure, efficient, and royalty-free cipher
- ▶ **The Finalists (1999):** Out of 15 initial submissions, 5 finalists were selected: MARS, RC6, **Rijndael**, Serpent, and Twofish.
- ▶ **The Winner (2000): Rijndael**, designed by Belgian cryptographers Joan Daemen and Vincent Rijmen
 - ▷ Excellent balance of security and performance
 - ▷ Flexibility across hardware and software
- ▶ **Standardization (2001):** Officially published as FIPS PUB 197

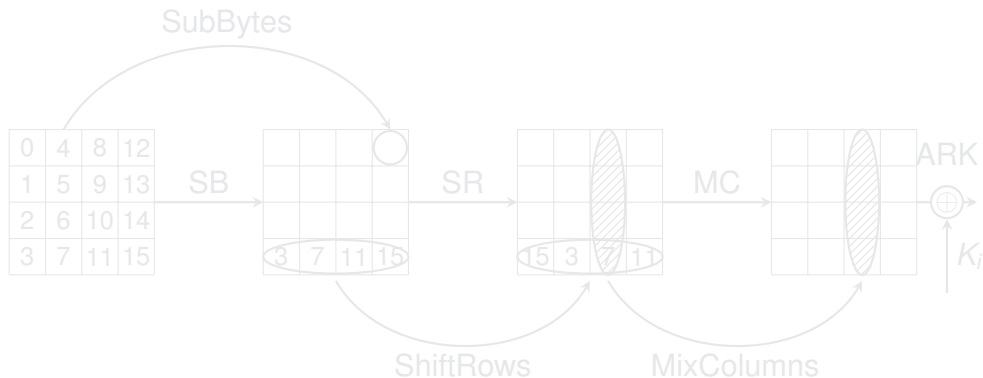
Round Function of AES

- ▶ AES has an SP (substitution-permutation) network structure.
- ▶ Block size: $n = 128$ bits, i.e., $P \in \{0, 1\}^{128}$ or $P \in \mathbb{F}_2^{128}$ or $P \in (\mathbb{F}_{2^8})^{16}$
- ▶ Key size: $\kappa = 128, 192,$ or 256



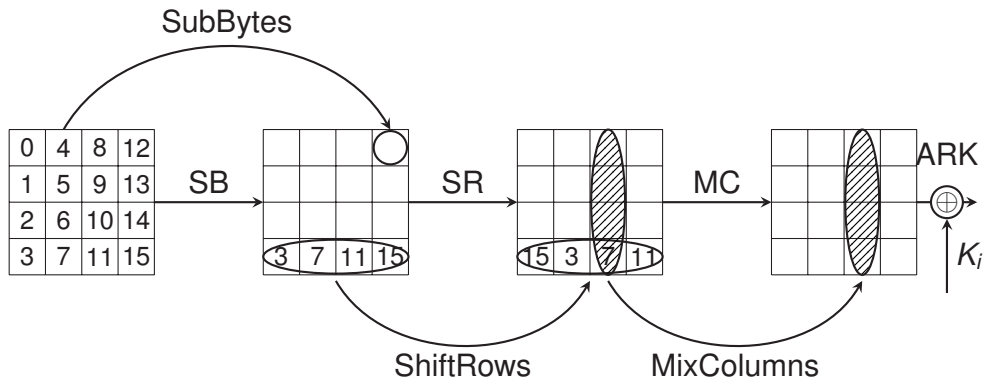
Round Function of AES

- ▶ AES has an SP (substitution-permutation) network structure.
- ▶ Block size: $n = 128$ bits, i.e., $\mathbf{P} \in \{0, 1\}^{128}$ or $\mathbf{P} \in \mathbb{F}_2^{128}$ or $\mathbf{P} \in (\mathbb{F}_{2^8})^{16}$
- ▶ Key size: $\kappa = 128, 192,$ or 256



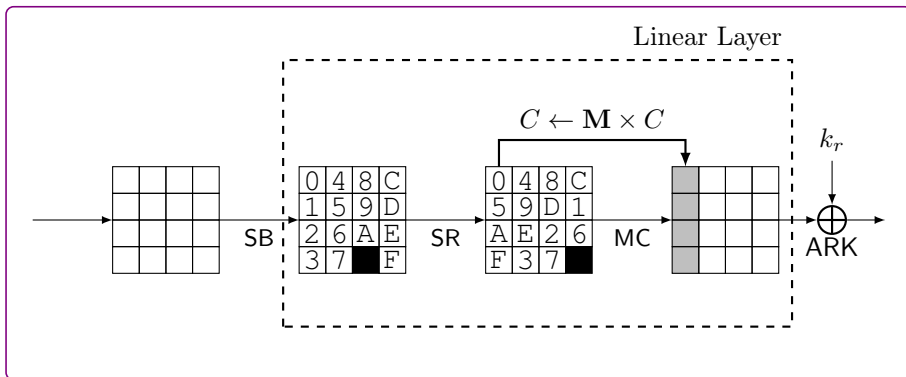
Round Function of AES

- ▶ AES has an SP (substitution-permutation) network structure.
- ▶ Block size: $n = 128$ bits, i.e., $\mathbf{P} \in \{0, 1\}^{128}$ or $\mathbf{P} \in \mathbb{F}_2^{128}$ or $\mathbf{P} \in (\mathbb{F}_{2^8})^{16}$
- ▶ Key size: $\kappa = 128, 192,$ or 256



Round Function of AES (cont.)

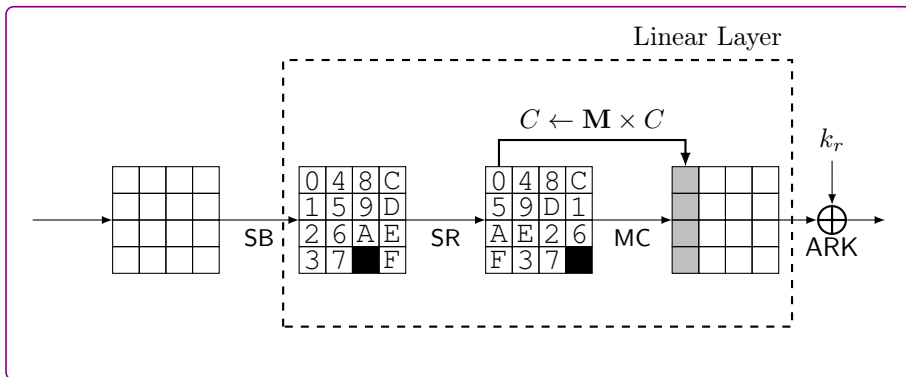
Round function



Number of rounds depends on the key length (10/12/14, respectively).

Round Function of AES (cont.)

Round function



Number of rounds depends on the key length (10/12/14, respectively).

The MixColumns Operation

- MixColumns treats each column of four bytes as four elements over $GF(2^8)$:

$$\begin{bmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{bmatrix}$$

- The irreducible polynomial of $GF(2^8)$ is $x^8 + x^4 + x^3 + x + 1$

The MixColumns Operation

- MixColumns treats each column of four bytes as four elements over $GF(2^8)$:

$$\begin{bmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{bmatrix}$$

- The irreducible polynomial of $GF(2^8)$ is $x^8 + x^4 + x^3 + x + 1$

“Galois Field with 2^8 elements”: Representation

- ▶ A state of AES is a sequence of bytes or elements of finite field $GF(2^8)$ or $\mathbb{F}_{2^8}$.
- ▶ A byte consists of 8 bits, i.e., binary string of length 8.
- ▶ Each element of $GF(2^8)$ which can be represented as a polynomial of degree up to 7 with coefficients are only 0 and 1

Integer	Binary	Polynomial in $GF(2^8)$
0	00000000	0
1	00000001	1
2	00000010	x
3	00000011	$x + 1$
16	00010000	x^4
21	00010101	$x^4 + x^2 + 1$

“Galois Field with 2^8 elements”: Representation

- ▶ A state of AES is a sequence of bytes or elements of finite field $GF(2^8)$ or $\mathbb{F}_{2^8}$.
- ▶ A byte consists of 8 bits, i.e., binary string of length 8.
- ▶ Each element of $GF(2^8)$ which can be represented as a polynomial of degree up to 7 with coefficients are only 0 and 1

Integer	Binary	Polynomial in $GF(2^8)$
0	00000000	0
1	00000001	1
2	00000010	x
3	00000011	$x + 1$
16	00010000	x^4
21	00010101	$x^4 + x^2 + 1$

“Galois Field with 2^8 elements”: Representation

- ▶ A state of AES is a sequence of bytes or elements of finite field $GF(2^8)$ or $\mathbb{F}_{2^8}$.
- ▶ A byte consists of 8 bits, i.e., binary string of length 8.
- ▶ Each element of $GF(2^8)$ which can be represented as a polynomial of degree up to 7 with coefficients are only 0 and 1

Integer	Binary	Polynomial in $GF(2^8)$
0	00000000	0
1	00000001	1
2	00000010	x
3	00000011	$x + 1$
16	00010000	x^4
21	00010101	$x^4 + x^2 + 1$

Addition in $GF(2^8)$

- ▶ Addition in a finite field works by adding the underlying polynomials
- ▶ No carry bits are generated as coefficients are modulo 2, $1 + 1 = 0$
- ▶ Therefore, addition, subtraction, and **XOR** ($\oplus$) are the exact same operation!

Example 1: addition

$$\begin{aligned}1 + 2 &\equiv (1) + (x) \\&= x + 1 \\&\equiv 00000011_2 \\&= 3\end{aligned}$$

Example 2: Binary wraparound

$$\begin{aligned}2 + 2 &\equiv (x) + (x) \\&= 2x \\&\equiv 0x \quad (\text{Since } 2 \bmod 2 = 0) \\&= 0\end{aligned}$$

Addition in $GF(2^8)$

- ▶ Addition in a finite field works by adding the underlying polynomials
- ▶ No carry bits are generated as coefficients are modulo 2, $1 + 1 = 0$
- ▶ Therefore, addition, subtraction, and **XOR** ($\oplus$) are the exact same operation!

Example 1: addition

$$\begin{aligned}1 + 2 &\equiv (1) + (x) \\&= x + 1 \\&\equiv 00000011_2 \\&= \mathbf{3}\end{aligned}$$

Example 2: Binary wraparound

$$\begin{aligned}2 + 2 &\equiv (x) + (x) \\&= 2x \\&\equiv 0x \quad (\text{Since } 2 \bmod 2 = 0) \\&= \mathbf{0}\end{aligned}$$

Multiplication in $GF(2^8)$

- ▶ Multiplication works just like standard polynomial multiplication.
- ▶ **Catch:** If the resulting degree is ≥ 8 , it falls outside $GF(2^8)$.
- ▶ **Fix:** Replace x^8 using the *irreducible polynomial*: $\mathbf{x^4 + x^3 + x + 1}$.

Examples

▶ $2 \times 2 \equiv (x) \times (x) = x^2 \equiv 00000100_2 = 4$

▶ What is 16×16 ? $16 \times 16 \equiv (x^4) \times (x^4) = x^8$

$$\begin{aligned} x^8 &\equiv x^8 \oplus (x^8 + x^4 + x^3 + x + 1) = x^4 + x^3 + x + 1 \\ &\equiv 00011011_2 = 27 \end{aligned}$$

Multiplication in $GF(2^8)$

- ▶ Multiplication works just like standard polynomial multiplication.
- ▶ **Catch:** If the resulting degree is ≥ 8 , it falls outside $GF(2^8)$.
- ▶ **Fix:** Replace x^8 using the *irreducible polynomial*: $\mathbf{x^4 + x^3 + x + 1}$.

Examples

▶ $2 \times 2 \equiv (x) \times (x) = x^2 \equiv 00000100_2 = 4$

▶ What is 16×16 ? $16 \times 16 \equiv (x^4) \times (x^4) = x^8$

$$\begin{aligned} x^8 &\equiv x^8 \oplus (x^8 + x^4 + x^3 + x + 1) = x^4 + x^3 + x + 1 \\ &\equiv 00011011_2 = 27 \end{aligned}$$

Multiplication in $GF(2^8)$

- ▶ Multiplication works just like standard polynomial multiplication.
- ▶ **Catch:** If the resulting degree is ≥ 8 , it falls outside $GF(2^8)$.
- ▶ **Fix:** Replace x^8 using the *irreducible polynomial*: $\mathbf{x^4 + x^3 + x + 1}$.

Examples

- ▶ $2 \times 2 \equiv (x) \times (x) = x^2 \equiv 00000100_2 = 4$
- ▶ What is 16×16 ? $16 \times 16 \equiv (x^4) \times (x^4) = \mathbf{x^8}$

$$\begin{aligned} x^8 &\equiv x^8 \oplus (x^8 + x^4 + x^3 + x + 1) = x^4 + x^3 + x + 1 \\ &\equiv 00011011_2 = \mathbf{27} \end{aligned}$$

Multiplication in $GF(2^8)$

- ▶ Multiplication works just like standard polynomial multiplication.
- ▶ **Catch:** If the resulting degree is ≥ 8 , it falls outside $GF(2^8)$.
- ▶ **Fix:** Replace x^8 using the *irreducible polynomial*: $\mathbf{x^4 + x^3 + x + 1}$.

Examples

- ▶ $2 \times 2 \equiv (x) \times (x) = x^2 \equiv 00000100_2 = 4$
- ▶ What is 16×16 ? $16 \times 16 \equiv (x^4) \times (x^4) = \mathbf{x^8}$

$$\begin{aligned}x^8 &\equiv x^8 \oplus (x^8 + x^4 + x^3 + x + 1) = x^4 + x^3 + x + 1 \\ &\equiv 00011011_2 = \mathbf{27}\end{aligned}$$

Multiplication by 2 and 3 in $GF(2^8)$

► Multiply by 2 (Polynomial x):

- ▷ **Left Shift:** Left shift the binary string by one position
- ▷ **Reduce:** If the leftmost bit (MSB) is 1, XOR the result with the reducing polynomial (00011011_2).

► Multiply by 3 (Polynomial $x + 1$):

- ▷ **Action:** Multiply by 2 (shift & reduce), then XOR with the original value.

Example: 27×2

27 in binary: 00011011_2

1. Left shift by 1:

$00011011_2 \ll 1 = 00110110_2$

2. Reduce? No

Result: $00110110_2 = 54$

Example: 27×3

Formula: $(27 \times 2) \oplus 27$

From the previous step:

$27 \times 2 = 00110110_2$

XOR with original value (27):

00110110_2 (54)

$\oplus 00011011_2$ (27)

00101101_2 (45)

Multiplication by 2 and 3 in $GF(2^8)$

► Multiply by 2 (Polynomial x):

- ▷ **Left Shift:** Left shift the binary string by one position
- ▷ **Reduce:** If the leftmost bit (MSB) is 1, XOR the result with the reducing polynomial (00011011_2).

► Multiply by 3 (Polynomial $x + 1$):

- ▷ **Action:** Multiply by 2 (shift & reduce), then XOR with the original value.

Example: 27×2

27 in binary: 00011011_2

1. Left shift by 1:

$$00011011_2 \ll 1 = 00110110_2$$

2. Reduce? No

Result: $00110110_2 = 54$

Example: 27×3

Formula: $(27 \times 2) \oplus 27$

From the previous step:

$$27 \times 2 = 00110110_2$$

XOR with original value (27):

$$\begin{array}{r} 00110110_2 \text{ (54)} \\ \oplus 00011011_2 \text{ (27)} \\ \hline 00101101_2 \text{ (45)} \end{array}$$

Multiplication by 2 as a Matrix Operation

- Let the byte be a column vector $\mathbf{b} = [b_0, b_1, b_2, b_3, b_4, b_5, b_6, b_7]^T$, where b_0 is the LSB (x^0) and b_7 is the MSB (x^7).

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ \textcolor{red}{1} & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & \textcolor{red}{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \textcolor{red}{1} & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \textcolor{red}{1} & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & \textcolor{red}{1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \textcolor{red}{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \textcolor{red}{1} & 0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix}$$

- **Sub-diagonal of 1s:** Performs the standard left shift ($b'_i = b_{i-1}$).
- **Rightmost Column:** Adds the reduction polynomial only when the MSB (b_7)

The SubBytes Operation

- ▶ Given input x , compute $y = x^{-1}$ (over the same field, with $0 \triangleq 0^{-1}$).
- ▶ Then compute the output as:

$$\begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \\ z_4 \\ z_5 \\ z_6 \\ z_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

AES SubBytes Operation as a Table Lookup

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
10	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
20	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
30	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
40	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
50	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
60	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
70	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
80	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
90	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A0	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B0	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C0	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D0	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E0	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F0	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Procedure: AES_Encrypt (P, K)

Input: 16-byte array P, Master-Key K

$State \leftarrow P$

$W \leftarrow \text{KeyExpansion}(K)$

$\text{AddRoundKey}(State, W[0])$

for $round \leftarrow 1$ **to** $N_r - 1$ **do**

$\text{SubBytes}(State)$

$\text{ShiftRows}(State)$

$\text{MixColumns}(State)$

$\text{AddRoundKey}(State, W[round])$

$\text{SubBytes}(State)$

$\text{ShiftRows}(State)$

$\text{AddRoundKey}(State, W[N_r])$

return out (C)

// Initialize 4×4 byte state matrix

// Generate round keys

// Initial AddRoundKey

// $N_r = 10, 12$, or 14

// Final round (No MixColumns)

Procedure: AES_Decrypt(C, K)

Input: 16-byte array C , Master-Key K

$State \leftarrow C$

$W \leftarrow \text{KeyExpansion}(K)$

$\text{AddRoundKey}(State, W[N_r])$

for $round \leftarrow N_r - 1$ **down to** 1 **do**

$\text{InvShiftRows}(State)$

$\text{InvSubBytes}(State)$

$\text{AddRoundKey}(State, W[round])$

$\text{InvMixColumns}(State)$

$\text{InvShiftRows}(State)$

$\text{InvSubBytes}(State)$

$\text{AddRoundKey}(State, W[0])$

return $State$ (P)

// Initialize state with the ciphertext

// Generate same round keys as encryption

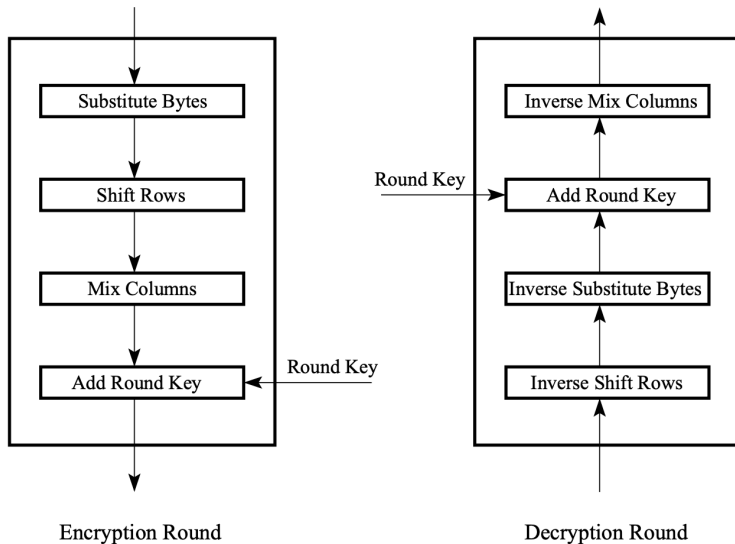
// Use the last round key first

// Iterate backwards

// Final round (No InvMixColumns)

// Use the first round key last

Comparison of Encryption and Decryption



AES Key Schedule Algorithm

Master-Key size = $32 \cdot Nk$ -bit key (e.g., $Nk = 4$ for AES-128)

- ▶ Initialize: $W[0, \dots, Nk - 1] = K[0, \dots, Nk - 1]$.
- ▶ For $i = Nk, \dots, 4 \cdot (7 + Nk) - 1$ do
 - ▷ If $i \equiv 0 \pmod{Nk}$ then
$$W[i] = W[i - Nk] \oplus \text{SB}(W[i - 1] \lll 8) \oplus \text{RCON}[i/Nk],$$
 - ▷ Else if $Nk \equiv 8$ and $i \equiv 4 \pmod{8}$ then
$$W[i] = W[i - 8] \oplus \text{SB}(W[i - 1]),$$
 - ▷ Otherwise
$$W[i] = W[i - 1] \oplus W[i - Nk],$$
- ▶ The first subkey is $W[0, 1, 2, 3]$, the second is $W[4, 5, 6, 7]$, etc.

AES Key Schedule Algorithm

Master-Key size = $32 \cdot Nk$ -bit key (e.g., $Nk = 4$ for AES-128)

- ▶ Initialize: $W[0, \dots, Nk - 1] = K[0, \dots, Nk - 1]$.
- ▶ For $i = Nk, \dots, 4 \cdot (7 + Nk) - 1$ do
 - ▷ If $i \equiv 0 \pmod{Nk}$ then
$$W[i] = W[i - Nk] \oplus \text{SB}(W[i - 1] \lll 8) \oplus \text{RCON}[i/Nk],$$
 - ▷ Else if $Nk \equiv 8$ and $i \equiv 4 \pmod{8}$ then
$$W[i] = W[i - 8] \oplus \text{SB}(W[i - 1]),$$
 - ▷ Otherwise
$$W[i] = W[i - 1] \oplus W[i - Nk],$$
- ▶ The first subkey is $W[0, 1, 2, 3]$, the second is $W[4, 5, 6, 7]$, etc.

AES Key Schedule Algorithm

Master-Key size = $32 \cdot Nk$ -bit key (e.g., $Nk = 4$ for AES-128)

- ▶ Initialize: $W[0, \dots, Nk - 1] = K[0, \dots, Nk - 1]$.
- ▶ For $i = Nk, \dots, 4 \cdot (7 + Nk) - 1$ do
 - ▷ If $i \equiv 0 \pmod{Nk}$ then
$$W[i] = W[i - Nk] \oplus \text{SB}(W[i - 1] \lll 8) \oplus \text{RCON}[i/Nk],$$
 - ▷ Else if $Nk \equiv 8$ and $i \equiv 4 \pmod{8}$ then
$$W[i] = W[i - 8] \oplus \text{SB}(W[i - 1]),$$
 - ▷ Otherwise
$$W[i] = W[i - 1] \oplus W[i - Nk],$$
- ▶ The first subkey is $W[0, 1, 2, 3]$, the second is $W[4, 5, 6, 7]$, etc.

AES Key Schedule Algorithm

Master-Key size = $32 \cdot Nk$ -bit key (e.g., $Nk = 4$ for AES-128)

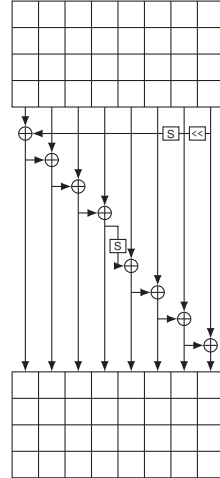
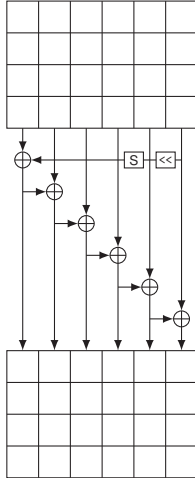
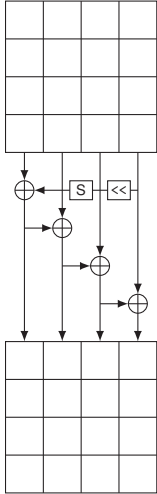
- ▶ Initialize: $W[0, \dots, Nk - 1] = K[0, \dots, Nk - 1]$.
- ▶ For $i = Nk, \dots, 4 \cdot (7 + Nk) - 1$ do
 - ▷ If $i \equiv 0 \pmod{Nk}$ then
$$W[i] = W[i - Nk] \oplus \text{SB}(W[i - 1] \lll 8) \oplus \text{RCON}[i/Nk],$$
 - ▷ Else if $Nk \equiv 8$ and $i \equiv 4 \pmod{8}$ then
$$W[i] = W[i - 8] \oplus \text{SB}(W[i - 1]),$$
 - ▷ Otherwise
$$W[i] = W[i - 1] \oplus W[i - Nk],$$
- ▶ The first subkey is $W[0, 1, 2, 3]$, the second is $W[4, 5, 6, 7]$, etc.

AES Key Schedule Algorithm

Master-Key size = $32 \cdot Nk$ -bit key (e.g., $Nk = 4$ for AES-128)

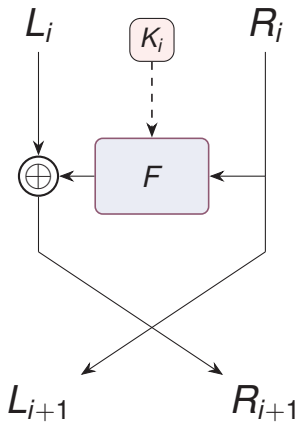
- ▶ Initialize: $W[0, \dots, Nk - 1] = K[0, \dots, Nk - 1]$.
- ▶ For $i = Nk, \dots, 4 \cdot (7 + Nk) - 1$ do
 - ▷ If $i \equiv 0 \pmod{Nk}$ then
$$W[i] = W[i - Nk] \oplus \text{SB}(W[i - 1] \lll 8) \oplus \text{RCON}[i/Nk],$$
 - ▷ Else if $Nk \equiv 8$ and $i \equiv 4 \pmod{8}$ then
$$W[i] = W[i - 8] \oplus \text{SB}(W[i - 1]),$$
 - ▷ Otherwise
$$W[i] = W[i - 1] \oplus W[i - Nk],$$
- ▶ The first subkey is $W[0, 1, 2, 3]$, the second is $W[4, 5, 6, 7]$, etc.

AES Key Schedule



The Feistel Structure and DES

The Feistel Structure

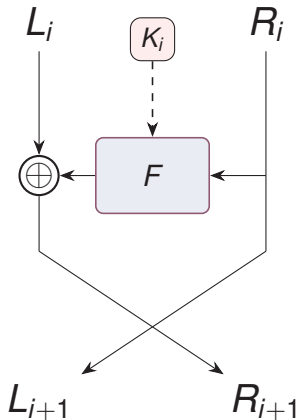


Encryption Round

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(K_i, R_i)$$

Exploring Weak Functions



Case 1: The Zero Function

What if $F(K, x) = 0 \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus 0 = L_0$$

$$L_2 = L_0$$

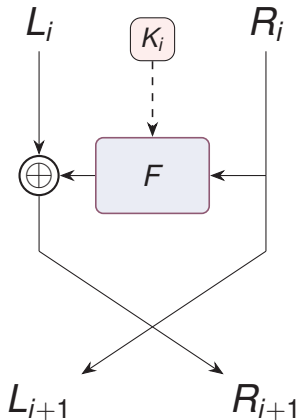
$$R_2 = R_0$$

Result:

Encryption of $L_0 \| R_0$ yields $L_0 \| R_0$

Decryption of $L_0 \| R_0$

Exploring Weak Functions



Case 1: The Zero Function

What if $F(K, x) = 0 \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus 0 = L_0$$

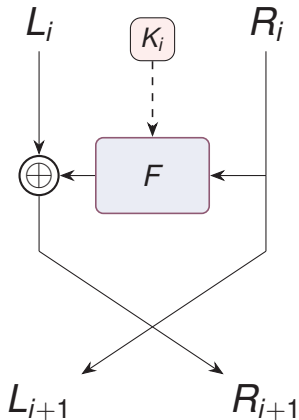
$$L_2 = L_0$$

$$R_2 = R_0$$

Result

After an even number of rounds, you get the input back. **No security!**

Exploring Weak Functions



Case 1: The Zero Function

What if $F(K, x) = 0 \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus 0 = L_0$$

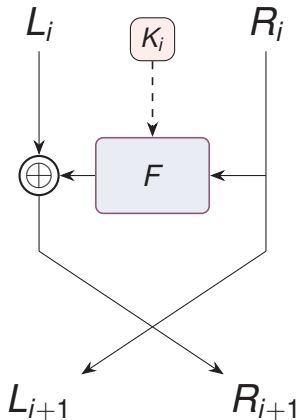
$$L_2 = L_0$$

$$R_2 = R_0$$

Result

After an even number of rounds, you get the input back. **No security!**

Exploring Weak Functions



Case 1: The Zero Function

What if $F(K, x) = 0 \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus 0 = L_0$$

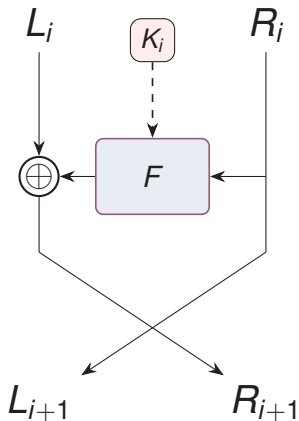
$$L_2 = L_0$$

$$R_2 = R_0$$

Result

After an even number of rounds, you get the input back. **No security!**

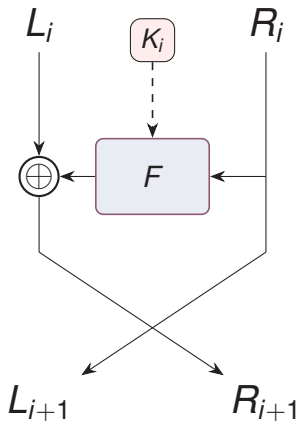
Exploring Weak Functions



Case 2: The Identity Function

What if $F(K, x) = x, \forall x$?

Exploring Weak Functions



Case 2: The Identity Function

What if $F(K, x) = x, \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0$$

$$L_2 = L_0 \oplus R_0$$

$$R_2 = R_0 \oplus (L_0 \oplus R_0) = L_0$$

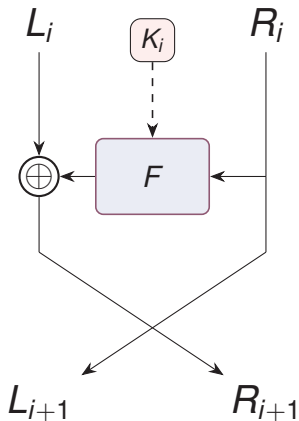
$$L_3 = L_0$$

$$R_3 = (L_0 \oplus R_0) \oplus L_0 = R_0$$

Result

The input (L_0, R_0) is restored after just 3 rounds. **No security!**

Exploring Weak Functions



Case 2: The Identity Function

What if $F(K, x) = x, \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0$$

$$L_2 = L_0 \oplus R_0$$

$$R_2 = R_0 \oplus (L_0 \oplus R_0) = L_0$$

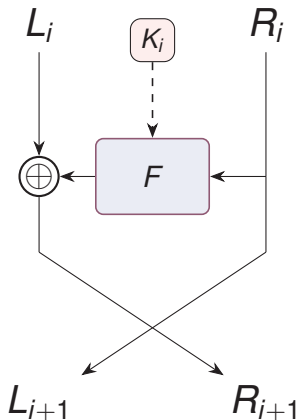
$$L_3 = L_0$$

$$R_3 = (L_0 \oplus R_0) \oplus L_0 = R_0$$

Result

The input (L_0, R_0) is restored after just 3 rounds. **No security!**

Exploring Weak Functions



Case 2: The Identity Function

What if $F(K, x) = x, \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0$$

$$L_2 = L_0 \oplus R_0$$

$$R_2 = R_0 \oplus (L_0 \oplus R_0) = L_0$$

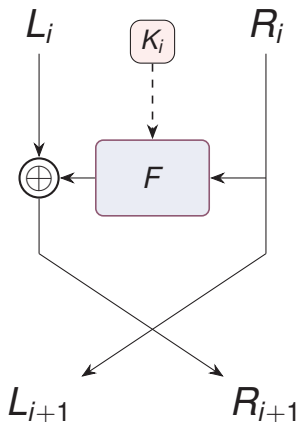
$$L_3 = L_0$$

$$R_3 = (L_0 \oplus R_0) \oplus L_0 = R_0$$

Result

The input (L_0, R_0) is restored after just 3 rounds. **No security!**

Exploring Weak Functions



Case 3: Simple XOR with Key

What if $F(K, x) = K \oplus x \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0 \oplus K_1$$

$$L_2 = L_0 \oplus R_0 \oplus K_1$$

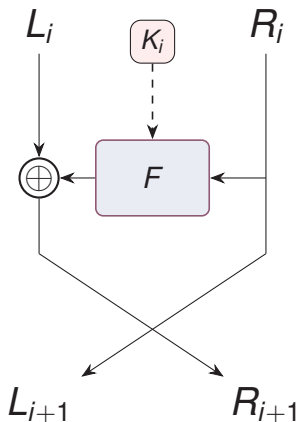
$$R_2 = R_0 \oplus (L_0 \oplus L_2) \oplus K_2$$

$$= L_0 \oplus K_1 \oplus K_2$$

Result

The output is just a linear function of the input and keys. Broken with basic algebra. No security!

Exploring Weak Functions



Case 3: Simple XOR with Key

What if $F(K, x) = K \oplus x \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0 \oplus K_1$$

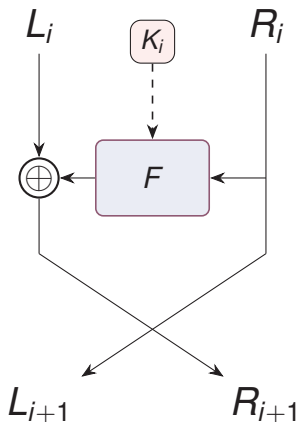
$$L_2 = L_0 \oplus R_0 \oplus K_1$$

$$\begin{aligned} R_2 &= R_0 \oplus (L_0 \oplus L_2) \oplus K_2 \\ &= L_0 \oplus K_1 \oplus K_2 \end{aligned}$$

Result

The output is just a linear function of the input and keys. Broken with basic algebra. No security!

Exploring Weak Functions



Case 3: Simple XOR with Key

What if $F(K, x) = K \oplus x \forall x$?

$$L_1 = R_0$$

$$R_1 = L_0 \oplus R_0 \oplus K_1$$

$$L_2 = L_0 \oplus R_0 \oplus K_1$$

$$R_2 = R_0 \oplus (L_0 \oplus L_2) \oplus K_2$$

$$= L_0 \oplus K_1 \oplus K_2$$

Result

The output is just a linear function of the input and keys. Broken with basic algebra. **No security!**

Conclusion: The Round Function is Everything

Main Lesson

A Feistel network is excellent **structure**, but its **round function** is everything.

- ▶ Weak or simple F functions lead to ciphers that are trivially broken.
- ▶ A strong F function must be **non-linear**

Conclusion: The Round Function is Everything

Main Lesson

A Feistel network is excellent **structure**, but its **round function** is everything.

- ▶ Weak or simple F functions lead to ciphers that are trivially broken.
- ▶ A strong F function must be **non-linear**

Conclusion: The Round Function is Everything

Main Lesson

A Feistel network is excellent **structure**, but its **round function** is everything.

- ▶ Weak or simple F functions lead to ciphers that are trivially broken.
- ▶ A strong F function must be **non-linear**



Thank You for your
attention!

Any questions?